A

Mini Project Report

On

HEALTH CARE Dashboard with Storytelling

Submitted to DR.K.V.SUBBA REDDY INSTITUTE OF TECHNOLOGY

In Partial Fulfillment of the requirements for the Award of Degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING (AIML)

Submitted By

NAME:- J SHIVA KRISHNA

ROLL NO:- 24FH1A3336

Under the Esteemed guidance of

Dr. B.MAHESH

Associate Professor, Department of CSE



Department of CSE(AIML)

DR.K.V.SUBBA REDDY INSTITUTE OF TECHNOLOGY

UGC AUTONOMOUS

(Approved by AICTE, NEW DELHI, Affiliated to JNTUA, Anantapur)

2026-2027

DR.K.V.SUBBA REDDY INSTITUTE OF TECHNOLOGY

UGC AUTONOMOUS

(Accredited by NBA,Approved by AICTE NEW DELHI, Affiliated to JNTUA, Anantapur)

Department of CSE(AIML)

CERTIFICATE

This is to certify that the project entitled **"Healthcare Dashboard with Storytelling"** is a bonafide work carried out by in partial fulfillment of the requirement for the award of the degree of **BACHELOR OF TECHNOLOGY** in **CSE(AIML)** from Dr.K.V.Subba Reddy Institute of Technology, Kurnool, under our guidance and supervision.

The results presented in this Mini project have been verified and are found to be satisfactory. The results embodied in this Mini project have not been submitted to any other university for the award of any other degree or diploma.

Guide **Head of the Department**

DECLARATION

This is to certify that the work reported in the present Mini project entitled "**Healthcare Dashboard with Storytelling"** is a record of bonafide work done by us in the Department of CSE(AI/AIML), Dr.K.V.Subba Reddy Institute of Technology, Kurnool. The reports are based on the project work done entirely by us and not copied from any other source. We submit our project for further development by any interested students who share similar interests to improve the project in the future.

The results embodied in this Mini project report have not been submitted to any other University or Institute for the award of any degree or diploma to the best of our knowledge and belief.

NAME:- J SHIVA KRISHNA

ROLL NO:- 24FH1A3336

ACKNOWLEDGMENT

We are extremely grateful to Dr. K Sree Latha, Principal and H. Ateeq Ahmed, HOD, Department of CSE(AIML), Dr.K.V.Subba Reddy Institute of Technology for their constant support. We are extremely thankful to Dr. B.Mahesh, Associate Professor, Department of CSE, Internal Guide, for his constant guidance, encouragement and moral support throughout the project.

We will be failing in duty if We do not acknowledge with grateful thanks to the authors of the references and other literatures referred in this Project.

We express our thanks to all staff members and friends for all the help and coordination extended in bringing out this project successfully in time.

Finally, we are very much thankful to our parents who guided us for every step.

INDEX

s.no	CONTENT	PAGE NO:
	ABSTRACT	I
1	INTRODUCTION	1
1.1	Need for the Project	2
1.2	Problem Statement	2
1.3	Scope	3
2	PROJECT OVERVIEW AND OBJECTIVES	4
2.1	Main Objectives	4
2.2	Expected Outcome	5
3	EXISTING SYSTEM AND PROPOSED SYSTEM	6
3.1	Existing System	6
3.2	Limitations of the Existing Approach	7
3.3	Proposed System	7
3.4	Proposed System Benefits	8
4	TECHNOLOGIES AND TOOLS USED	9
4.1	Why Python?	10
4.2	Why Streamlit?	10
5	DATASET DESCRIPTION	11
5.1	Data Loading	12
5.2	Data Quality Considerations	12
6	SYSTEM REQUIREMENTS	13
6.1	Hardware Requirements	13
6.2	Software Requirements	14
6.3	Python Libraries	14
7	SYSTEM DESIGN AND ARCHITECTURE	15

7.1	Data Flow	16
8	IMPLEMENTATION AND WORKING PROCEDURE	17
8.1	Starting the Application	17
8.2	Data Preparation	18
8.3	Applying Filters	18
8.4	Exporting Results	19
9	DASHBOARD MODULES AND DATA STORYTELLING	20
9.1	Storytelling Sequence	22
9.2	Example of a Careful Explanation	22
10	DATA VISUALISATION AND ANALYSIS	23
10.1	Key Performance Indicators	23
10.2	Interpreting Trends	24
10.3	Comparing Departments	24
10.4	Financial Measures	25
11	RESULTS AND DISCUSSION	26
12	ADVANTAGES AND LIMITATIONS	27
12.1	Advantages	27
12.2	Limitations	28
13	PRIVACY, ETHICS AND SAFETY	29
14	FUTURE ENHANCEMENTS	30
15	CONCLUSION	31
16	REFERENCES	32
17	APPENDIX	33
17.1	Suggested Project Folder Structure	33
17.2	Windows CMD / VS Code Run Procedure	34
17.3	Sample Storytelling Questions	35

ABSTRACT

Healthcare organisations produce information every day through patient visits, department services, diagnoses, treatments, length of stay and billing records. When these details are stored only as rows in a spreadsheet, it may take time for a person to understand the overall situation. A dashboard can make the information easier to explore by presenting important values and patterns in a visual form.

This mini project, titled “Healthcare Data Storytelling Dashboard Using Python and Streamlit”, is developed as an educational data analysis application. It provides an interface for viewing healthcare visit information, filtering records and studying patterns through charts. The application includes an in-built fictional dataset of 900 sample visits and also supports uploading a CSV file that follows the required column structure.

The dashboard presents an executive overview, visit trends, department insights, patient and treatment analysis, and records with export options. It uses Python with Pandas and NumPy for data handling, Streamlit for the user interface, Plotly for interactive visualisations and Matplotlib for PDF chart output. Users can review total visits, sample patient counts, average age, average length of stay and bill-related measures.

The main purpose is to demonstrate how data storytelling can turn a collection of records into a sequence of understandable observations. The dashboard is not a medical diagnosis system and does not recommend treatments. Its built-in data is synthetic, and any uploaded data must be authorised and de-identified before use.

1. INTRODUCTION

Data is an important part of modern healthcare administration. Hospitals and clinics may maintain records about the number of visits, departments, patient age groups, types of visits, diagnosis categories, treatments, outcomes and billing amounts. These records can help administrators understand service usage and plan resources. However, raw tables do not always make patterns immediately visible, especially when the number of records becomes large.

Data visualisation is the process of representing information through charts, graphs and other visual elements. A well-designed dashboard brings related measures together on one screen. It allows a user to move from a general view to a more detailed view by applying filters or selecting categories. This can make routine analysis more convenient and can support discussions based on the available data.

Data storytelling adds an explanation to the visual presentation. Instead of displaying a chart without context, the user begins with a question, observes the pattern, explains what the data shows and identifies what should be checked next. For example, a chart showing visits by month may help the user notice whether the number of visits changes over time. The chart alone does not explain the reason for the change. Further investigation would be needed before suggesting a cause.

The proposed project applies these ideas to a sample healthcare visit dataset. It is intended for students to practise data preparation, dashboard development, chart interpretation and report writing. The project uses a simple web interface so that the user can interact with the information through a browser.

1.1 Need for the Project

A structured dashboard is useful when a user needs to review several measures together. Without a dashboard, the user may have to open different sheets, apply filters manually and create charts one by one. This can make repeated analysis slower. A dashboard places the main measures and visual summaries in a common interface and provides export options for further study.

1.2 Problem Statement

Healthcare visit information may be available in tabular form, but a table by itself may not clearly communicate trends, department comparisons or differences between visit categories. The problem addressed by this mini project is to develop a beginner-friendly application that organises healthcare visit data, allows filtering and displays important information through interactive visualisations and simple explanations.

1.3 Scope

The scope is limited to educational analysis of visit-level sample data. The project covers data loading, column validation, filtering, summary calculations, visualisation and file export. It does

not connect to a hospital management system, electronic health record platform or live patient database. It does not provide clinical recommendations, diagnosis, risk prediction or treatment decisions.

2. PROJECT OVERVIEW AND OBJECTIVES

The Healthcare Data Storytelling Dashboard is a local Python application that runs through Streamlit. When the application starts, it prepares a sample dataset and displays the dashboard in a web browser. A user can continue with the sample data or upload a CSV file containing the expected fields. After loading, the application provides filters and several sections for exploring the information.

2.1 Main Objectives

- To develop a simple dashboard for exploring healthcare visit records.
- To display key measures such as total visits, unique sample patients, average age and average length of stay.
- To analyse visit patterns over time and compare departments and visit categories.
- To provide interactive charts that make category comparisons easier to understand.
- To support CSV upload with checks for required columns.
- To allow filtered records and analysis outputs to be downloaded in common file formats.
- To practise data storytelling by explaining what a chart shows and what it does not prove.

2.2 Expected Outcome

The expected outcome is a working educational dashboard that can load the sample data, display summary measures, respond to user filters and provide visual reports. A student should be able to use the application to explain the structure of the dataset, identify visible patterns and describe the limitations of the analysis.

3. EXISTING SYSTEM AND PROPOSED SYSTEM

3.1 Existing System

In a basic manual approach, data may be stored in spreadsheets and reviewed by sorting, filtering or calculating totals. Charts can be created separately when required. This approach is suitable for small tasks, but repeated work may be needed when the date range or category changes. The user may also need to move between multiple sheets to compare different measures.

3.2 Limitations of the Existing Approach

- Repeated manual filtering and chart preparation can take additional time.
- Different summaries may be stored in separate locations.
- A user may find it difficult to compare several categories at once.
- Charts may not include a clear explanation of the question or the observation.
- Exporting a consistent set of results may require extra steps.

3.3 Proposed System

The proposed system combines data loading, filtering, summary calculations, charts and downloads in one dashboard. Streamlit provides the user interface, while Python libraries perform data processing and visualisation. The dashboard is divided into sections so that users can study the overall picture first and then move to trends, departments, patient and treatment categories, and individual records.

3.4 Proposed System Benefits

The proposed system reduces the need to prepare each chart separately. It also gives the user a repeatable way to explore the data. When filters are changed, the displayed summaries and charts can reflect the selected records. This makes the application suitable for classroom demonstrations and introductory data analytics practice.

4. TECHNOLOGIES AND TOOLS USED

Technology	Purpose
Python	Main programming language used to organise the application logic and work with data.
Streamlit	Python framework used to create the interactive dashboard interface.
Pandas	Library used for loading tabular data, selecting columns, filtering records and calculating summaries.
NumPy	Library used for numerical operations and supporting data calculations.
Plotly Express	Library used to create interactive charts that users can explore.
Matplotlib	Library used for chart rendering and PDF-oriented output.
OpenPyXL	Library used to support Excel workbook export.
Visual Studio Code	Code editor used to create, edit and run the Python project.
Web browser	Displays the local Streamlit dashboard after the application starts.

4.1 Why Python?

Python has a readable syntax and a wide collection of libraries for data analysis. It allows a student to perform data preparation and visualisation without writing every operation from the beginning. The use of commonly available libraries also makes it easier to extend the project

in later stages.

4.2 Why Streamlit?

Streamlit allows a Python program to display controls, text, tables and charts as a web application. It is useful for this project because the user can interact with the dashboard without developing a separate front-end application.

5. DATASET DESCRIPTION

The built-in dataset contains 900 fictional healthcare visit records. It is created for demonstration and learning. The sample dates range from 02 January 2024 to 31 December 2026. These records do not represent actual patients, a real hospital or verified medical events.

Field	Description
Visit Date	Date on which the sample visit is recorded.
Patient ID	Fictional identifier used to count sample patients.
Age	Age value recorded for the sample visit.
Gender	Gender category included in the demonstration data.
Department	Department associated with the visit.
Visit Type	Category describing the type of visit.
Diagnosis Group	Broad diagnosis category used for grouping.
Treatment	Treatment category recorded in the sample.
Length of Stay (Days)	Number of days associated with the sample stay.
Outcome	Recorded sample outcome category.
Bill Amount (INR)	Fictional bill amount represented in Indian rupees.

5.1 Data Loading

The application can use its built-in sample records or accept a CSV upload. A CSV file is a plain-text table format that can be exported from spreadsheet software. When a file is uploaded, the application checks whether the expected columns are available. If required columns are missing, the user receives an error message rather than continuing with an incomplete structure.

5.2 Data Quality Considerations

Before using a real dataset, the user should check date formats, missing values, duplicate rows, inconsistent category names and invalid numerical values. A value such as a negative length of stay or an incorrectly formatted date may affect the analysis. The project demonstrates the importance of validating the data before interpreting charts.

6. SYSTEM REQUIREMENTS

6.1 Hardware Requirements

- A desktop computer or laptop.
- A modern processor and sufficient memory to run Python and a web browser.
- Storage space for the source code, dataset and exported files.
- A display and keyboard for interacting with the dashboard.

6.2 Software Requirements

- Windows, macOS or Linux operating system; the development instructions in this project focus on Windows.
- Python 3.10 or later is recommended, subject to library compatibility.
- Visual Studio Code or another Python code editor.
- A web browser such as Chrome, Edge or Firefox.
- Required Python packages listed in requirements.txt.

6.3 Python Libraries

The project uses Streamlit, Pandas, NumPy, Plotly, Matplotlib and OpenPyXL. These packages are listed in the requirements file so that they can be installed together. Package versions and Python compatibility should be checked if an installation error occurs.

7. SYSTEM DESIGN AND ARCHITECTURE

The system follows a simple flow. First, the application loads the built-in sample data or reads the user's CSV file. Next, it checks the required fields and prepares the data for analysis. The user selects filters such as date range, department, visit type and outcome. The application then calculates summary values and creates charts based on the selected records. Finally, the user can view the results on screen or download supported outputs.

- Input layer: built-in sample data or user-provided CSV file.
- Validation layer: checks the expected column structure and prepares data types.
- Filtering layer: applies the selected date and category filters.
- Analysis layer: calculates totals, averages and grouped summaries.
- Visualisation layer: creates charts and tables for the dashboard sections.
- Export layer: prepares downloadable files for the user's selected results.

7.1 Data Flow

The data flow begins with a source file or sample dataset. The data is loaded into a Pandas DataFrame, which is a table-like structure used by the application. The selected filters produce a working subset of records. Summary calculations and chart groupings are then based on that subset. This approach helps keep the displayed results consistent with the selected filters.

8. IMPLEMENTATION AND WORKING PROCEDURE

8.1 Starting the Application

The project is intended to be run from its project folder. The user installs the required packages

and starts Streamlit from the terminal. Streamlit then provides a local address that can be opened in a web browser. The dashboard runs on the user's computer; it is not automatically a public website.

8.2 Data Preparation

After the source data is loaded, the application prepares the date and numerical columns for analysis. The required column validation helps ensure that later calculations can refer to the expected field names. The user should keep the column headings consistent when preparing a CSV file.

8.3 Applying Filters

Filters allow the user to focus on a selected period or category. For example, selecting a department can narrow the analysis to visits associated with that department. Selecting a date range can show only records within that period. When describing a chart, the report should mention the filters used because the displayed result depends on the selected records.

8.4 Exporting Results

The dashboard provides download options for common formats, including CSV, Excel, PDF and PNG outputs, depending on the selected export feature. CSV is useful for transferring tabular records. Excel is useful for further spreadsheet work. PDF is convenient for sharing a fixed report, while PNG is useful for inserting a chart into a presentation or document.

9. DASHBOARD MODULES AND DATA STORYTELLING

Executive Overview

Presents the main key performance indicators and an initial view of the selected dataset. It helps the user understand the scale of the records before examining details*Figure 1: Illustrative healthcare dashboard – executive overview*



Healthcare Visit Trends

Shows how visits are distributed over time. The user can look for increases, decreases or repeated patterns across the selected period.

Department Insights

Compares departments using the available visit records and related measures. It can help identify which categories have larger or smaller counts in the filtered data.

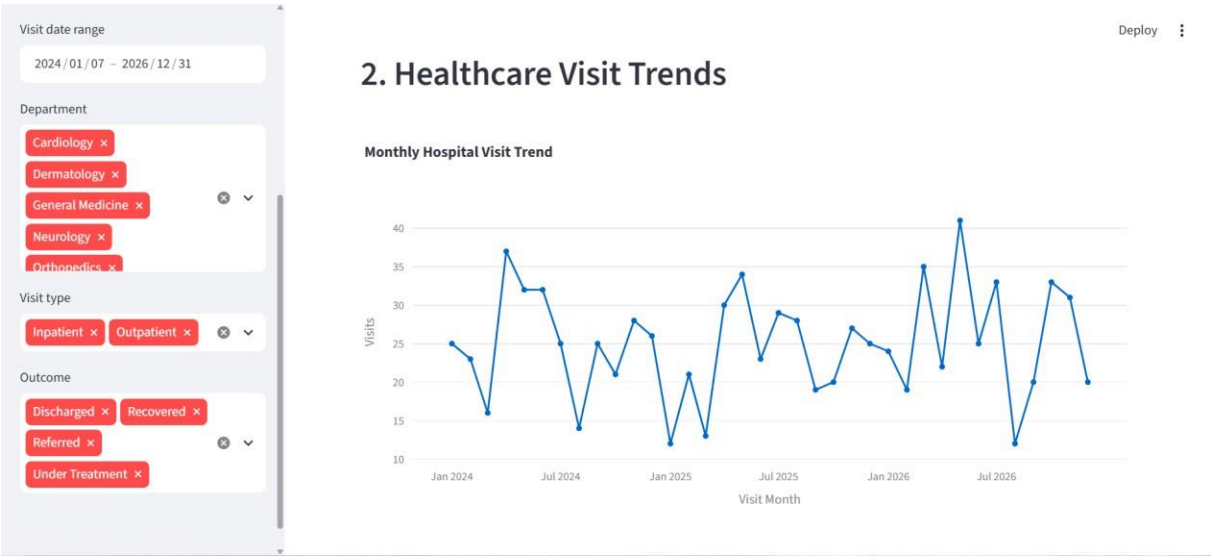


Figure 2: Illustrative healthcare visit trends and department comparison

Patient & Treatment Analysis

Explores sample patient characteristics, diagnosis groups, treatment categories and outcomes. These are descriptive summaries and do not establish clinical effectiveness.



Figure 3: Illustrative patient, treatment and records analysis

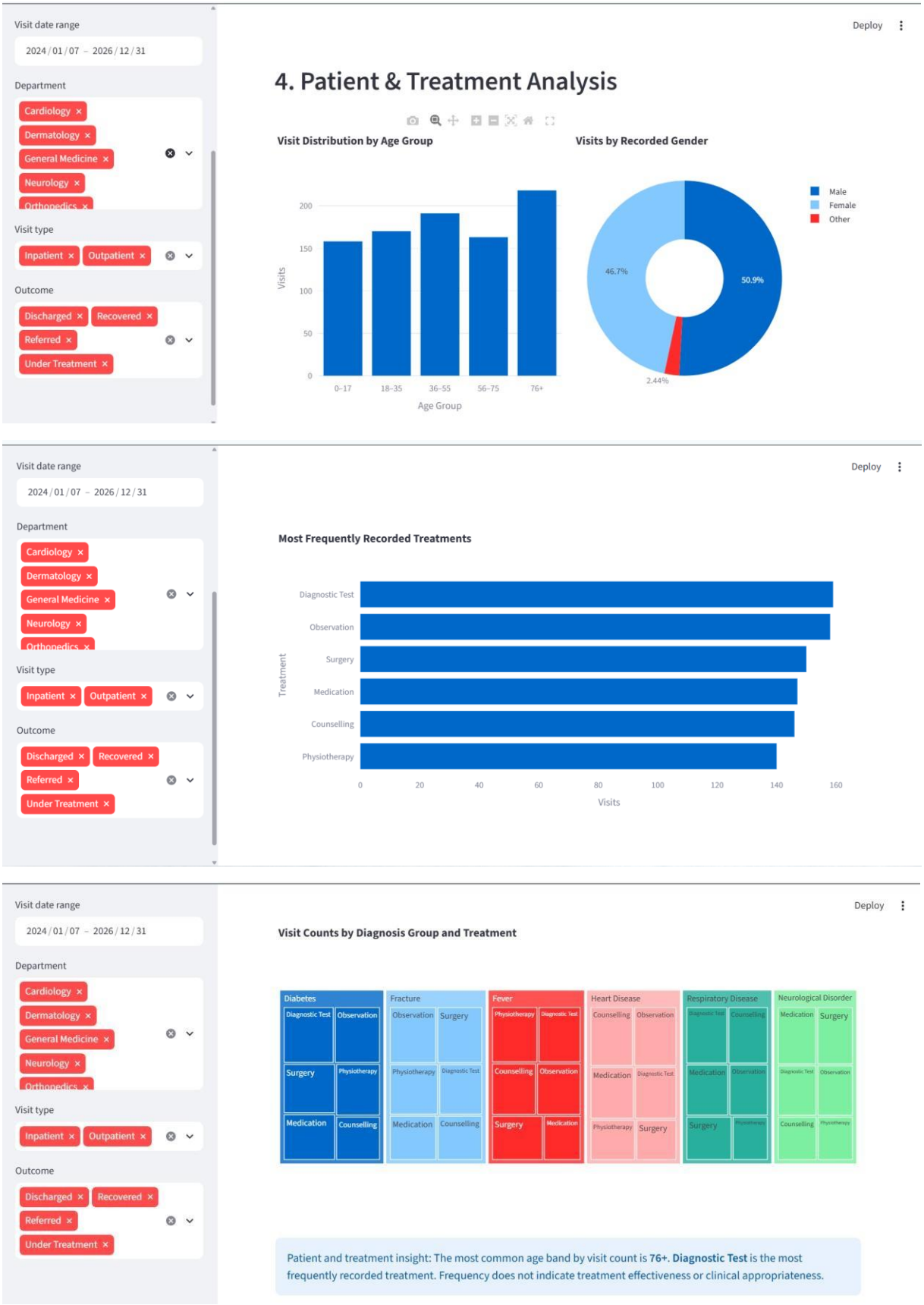


Figure 4: Patient & Treatment Analysis showing Visit Distribution by Age Group and Visits by Recorded Gender.

Records & Exports

Provides access to filtered records and available download options so that the user can keep a copy of the analysis.

9.1 Storytelling Sequence

A useful data story begins with a clear question. The user first introduces the dataset and the period being analysed. Next, the user presents the overall visit count and other summary measures. The story then moves to time trends, department comparisons and patient or treatment categories. At the end, the user summarises the observations and identifies information that would need further checking.

9.2 Example of a Careful Explanation

A suitable explanation could be: “The monthly chart shows that the number of recorded visits is higher in some months than in others. This is an observation from the selected dataset. The chart does not explain why the difference occurred. To understand the reason, additional information such as service availability, seasonal factors and changes in recording practices would be required.” This style separates what the data shows from a possible explanation.

10. DATA VISUALISATION AND ANALYSIS

Visualisation makes it easier to compare values and notice patterns. The correct chart depends on the question. A line chart is commonly used to show change over time. A bar chart is useful for comparing categories. A histogram can show how numerical values are distributed. A table is useful when the user needs to inspect exact records.

10.1 Key Performance Indicators

- Total visits: number of visit records included after applying the filters.
- Unique sample patients: number of distinct fictional patient identifiers in the selected records.
- Average age: mean age value in the filtered data.
- Average length of stay: mean number of stay days in the filtered records.
- Total bill amount: sum of the sample bill amounts.
- Average bill amount: mean bill amount for the selected records.

10.2 Interpreting Trends

When a trend chart is reviewed, the user should state the date range and any filters. The explanation should identify the visible movement and avoid claiming a cause without evidence. If a month has a larger count, it is correct to say that the chart records more visits for that month. It is not correct to conclude from the chart alone that a disease outbreak or a change in hospital quality caused the increase.

10.3 Comparing Departments

Department comparisons can show how the visit records are distributed among the available

department categories. A higher count indicates more records in that category within the selected dataset. It does not automatically mean that the department performs better or worse. Interpretation should consider department size, service type, operating hours and the completeness of the data.

10.4 Financial Measures

Bill amount measures describe the values stored in the sample records. They should not be interpreted as actual hospital revenue, patient affordability or treatment cost effectiveness. If real authorised data is used in a future version, the meaning of each financial field should be confirmed with the data owner.

11. RESULTS AND DISCUSSION

The completed dashboard provides a structured way to view the sample healthcare visit dataset. The user can begin with an overall summary and then move to different sections to examine time patterns, department categories, patient and treatment information, and individual records. Filters allow the user to narrow the displayed results, while export features make it possible to save selected information.

The built-in dataset contains 900 fictional visit records with dates spanning 02 January 2024 to 31 December 2026. Because the data is generated for demonstration, the charts show how the application works rather than reporting actual healthcare activity. The dashboard can be used to demonstrate the steps of loading data, validating columns, filtering records, calculating summaries and communicating observations.

The main result of this mini project is the integration of data preparation, interactive analysis and storytelling guidance in one application. The project also demonstrates that visual results must be interpreted carefully. A chart can reveal a pattern in the records, but it cannot by itself establish the reason for that pattern.

12. ADVANTAGES AND LIMITATIONS

12.1 Advantages

- Simple browser-based interface created with Streamlit.
- Provides several analysis sections in one application.
- Supports a built-in dataset for practice and CSV upload for structured data.
- Includes filters for exploring selected records.
- Uses interactive charts to support visual comparison.
- Provides export options for further study and report preparation.
- Encourages the user to explain observations rather than only display charts.

12.2 Limitations

- The built-in dataset is fictional and cannot be used to describe real hospital performance.
- The application depends on the quality and structure of the uploaded CSV file.
- Missing or incorrectly formatted values can affect calculations if not handled properly.

- The dashboard is descriptive; it does not predict future visits or patient outcomes.
- It does not provide diagnosis, treatment recommendations or clinical decision support.
- It does not connect to live hospital systems or verify the accuracy of external records.
- Results depend on the filters selected by the user.

13. PRIVACY, ETHICS AND SAFETY

Healthcare information can contain sensitive personal details. A student project should not use identifiable patient information without proper authorisation and safeguards. Names, phone numbers, addresses, medical record numbers and other direct identifiers should not be included in a demonstration dataset. If real data is approved for a future academic activity, it should be de-identified and handled according to the institution's rules and applicable privacy requirements.

The built-in dataset in this project is fictional. Users should not upload identifiable patient records to the dashboard. The application is an educational analytics demonstration and must not be used to diagnose a person, select a treatment or make an urgent medical decision. Any real healthcare analysis must be reviewed by qualified professionals and the responsible data owner.

14. FUTURE ENHANCEMENTS

The project can be improved in several ways after the basic dashboard has been tested. One possible enhancement is to add stronger data-quality reports that identify missing values, duplicate records and invalid ranges before analysis. Another improvement is to provide more flexible date grouping, such as weekly, monthly and yearly views.

- Add a configurable data validation page with clear warnings.
- Support additional authorised file formats where appropriate.
- Provide user-selectable chart types and more detailed chart explanations.
- Add a report builder that records the filters and date range used.
- Improve accessibility with clear labels, readable colours and keyboard-friendly controls.
- Add role-based access and secure storage only if the project is developed for an approved real-world environment.
- Connect to an approved database only after security, privacy and governance requirements are addressed.

Any future predictive feature would require a suitable, representative dataset, careful model evaluation and expert review. It should not be added merely to make the dashboard appear more advanced.

15. CONCLUSION

The Healthcare Data Storytelling Dashboard Using Python and Streamlit demonstrates how healthcare visit records can be organised and presented through an interactive application. The project combines data loading, column validation, filtering, summary calculations, charts and export options. Its five main dashboard sections guide the user from an overall view to

trends, department comparisons, patient and treatment categories, and detailed records.

The project also highlights the importance of data storytelling. A useful dashboard should not only display numbers; it should help the user ask a question, identify what the data shows and explain the limitations of the observation. The sample records used in this project are fictional, so the results are suitable for learning and demonstration only. They do not represent actual patient activity or provide medical guidance.

Overall, this mini project helped demonstrate practical use of Python libraries for data analysis and visualisation. It provides a foundation that can be extended with improved validation, reporting and secure data handling in future academic work.

16. REFERENCES

- Python Software Foundation, Python Documentation, https://docs.python.org/3/
- Streamlit Documentation, https://docs.streamlit.io/
- Pandas Documentation, https://pandas.pydata.org/docs/
- NumPy Documentation, https://numpy.org/doc/
- Plotly Python Documentation, https://plotly.com/python/
- Matplotlib Documentation, https://matplotlib.org/stable/
- OpenPyXL Documentation, https://openpyxl.readthedocs.io/
- User-provided reference PDF: “Sales Data Visualization Dashboard with Storytelling” (used as a structural and storytelling reference; sales-specific content adapted to the healthcare project).

17. APPENDIX

17.1 Suggested Project Folder Structure

```
Healthcare_Data_Storytelling_Project/
    app.py
    requirements.txt
    README.txt
    .vscode/
        launch.json
    Project_Report.docx
```

SOURCE CODE:

GITHUBRESPOSITORY:-https://github.com/shivakrishna3336-debug/Data-Visualization-Project/tree/main

17.2 Windows CMD / VS Code Run Procedure

1. Extract the project ZIP file to a folder, for example Documents\Healthcare_Data_Storytelling_Project.

2. Open Visual Studio Code and select File > Open Folder. Choose the extracted project folder.

3. Open Terminal > New Terminal in VS Code.

```
py -3.14 -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements.txt
python -m streamlit run app.py
```

4. Wait for Streamlit to display the local address, then open that address in a browser.

5. To stop the application, return to the terminal and press Ctrl+C.

Important: Run the application with the Streamlit command shown above. Do not use the VS Code Run Code button for this Streamlit project, because it may execute the file in a way that does not start the dashboard server.

17.3 Sample Storytelling Questions

- How many visit records are included in the current selection?
- Which months show higher or lower visit counts?
- How are the selected visit records distributed across departments?
- What age groups appear in the filtered records?
- How do the recorded visit types and outcomes compare?
- What additional information would be required before explaining a pattern?

End of Report

Complete source code (app.py)

```
import io

import matplotlib.pyplot as plt
from matplotlib.backends.backend_pdf import PdfPages
import numpy as np
import pandas as pd
import plotly.express as px
import streamlit as st

# ============================================================
# 1. PAGE CONFIGURATION
# ============================================================
st.set_page_config(
    page_title="Healthcare Data Storytelling Dashboard",
    page_icon="🏥",
    layout="wide"
)

REQUIRED_COLUMNS = [
    "Visit  Date",
    "Patient ID",
    "Age",
    "Gender",
    "Department",
    "Visit Type",
    "Diagnosis Group",
    "Treatment",
    "Length of Stay (Days)",
    "Outcome",
    "Bill Amount (INR)"
]

# ============================================================
# 2. SYNTHETIC HEALTHCARE SAMPLE DATA
# ============================================================
@st.cache_data
def load_sample_data():
    """Generate fictional educational healthcare visit records."""

    rng = np.random.default_rng(42)
    row_count = 900

    departments = [
        "Cardiology",
        "Neurology",
        "Orthopedics",
        "General Medicine",
        "Pediatrics",
        "Dermatology"
    ]

    diagnosis_groups = [
        "Heart Disease",
        "Diabetes",
        "Fever",
        "Fracture",
        "Respiratory Disease",
        "Neurological Disorder"
    ]

    treatments = [
        "Medication",
        "Surgery",
        "Physiotherapy",
```

```
        "Diagnostic Test",
        "Observation",
        "Counselling"
    ]

    visit_dates = rng.choice(
        pd.date_range("2024-01-02", "2026-12-31", freq="D"),
        size=row_count
    )

    patient_numbers = rng.integers(1, 351, size=row_count)

    sample = pd.DataFrame({
        "Visit Date": visit_dates,
        "Patient ID": [f"P{number:04d}" for number in patient_numbers],
        "Age": rng.integers(1, 100, size=row_count),
        "Gender": rng.choice(
            ["Male", "Female", "Other"],
            size=row_count,
            p=[0.49, 0.49, 0.02]
        ),
        "Department": rng.choice(departments, size=row_count),
        "Visit Type": rng.choice(
            ["Inpatient", "Outpatient"],
            size=row_count,
            p=[0.35, 0.65]
        ),
        "Diagnosis Group": rng.choice(diagnosis_groups, size=row_count),
        "Treatment": rng.choice(treatments, size=row_count),
        "Length of Stay (Days)": rng.integers(0, 16, size=row_count),
        "Outcome": rng.choice(
            ["Recovered", "Under Treatment", "Referred", "Discharged"],
            size=row_count,
            p=[0.35, 0.25, 0.10, 0.30]
        ),
        "Bill Amount (INR)": rng.integers(500, 50001, size=row_count)
    })

    return sample

# ============================================================
# 3. DATA VALIDATION AND PREPARATION
# ============================================================
def prepare_data(raw_data):
    df = raw_data.copy()
    df.columns = df.columns.astype(str).str.strip()

    missing_columns = [
        column for column in REQUIRED_COLUMNS
        if column not in df.columns
    ]

    if missing_columns:
        raise ValueError(
            "Missing required columns: " + ", ".join(missing_columns)
        )

    df["Visit Date"] = pd.to_datetime(
        df["Visit Date"],
        errors="coerce"
    )

    numeric_columns = [
        "Age",
        "Length of Stay (Days)",
        "Bill Amount (INR)"
    ]

    for column in numeric_columns:
        df[column] = pd.to_numeric(df[column], errors="coerce")
```

```
    df = df.dropna(subset=REQUIRED_COLUMNS).copy()

    df = df[
        (df["Age"] >= 0)
        & (df["Age"] <= 110)
        & (df["Length of Stay (Days)"] >= 0)
        & (df["Bill Amount (INR)"] >= 0)
    ].copy()

    if df.empty:
        raise ValueError(
            "No valid healthcare records remain after data cleaning."
        )

    df["Age Group"] = pd.cut(
        df["Age"],
        bins=[-1, 17, 35, 55, 75, 110],
        labels=["0–17", "18–35", "36–55", "56–75", "76+"]
    )

    df["Visit Month"] = (
        df["Visit Date"].dt.to_period("M").dt.to_timestamp()
    )

    return df

# ============================================================
# 4. HELPER FUNCTIONS
# ============================================================
def inr(value):
    return f"₹{value:,.0f}"

def make_excel_report(data, summary, monthly, department_summary,
                      treatment_summary):
    output = io.BytesIO()

    with pd.ExcelWriter(output, engine="openpyxl") as writer:
        data.to_excel(
            writer,
            index=False,
            sheet_name="Filtered Visits"
        )
        summary.to_excel(
            writer,
            index=False,
            sheet_name="KPI Summary"
        )
        monthly.to_excel(
            writer,
            index=False,
            sheet_name="Monthly Trends"
        )
        department_summary.to_excel(
            writer,
            index=False,
            sheet_name="Departments"
        )
        treatment_summary.to_excel(
            writer,
            index=False,
            sheet_name="Treatments"
        )

    output.seek(0)
    return output.getvalue()

def make_pdf_report(data, monthly, department_summary, treatment_summary):
    output = io.BytesIO()

    total_visits = len(data)
    unique_patients = data["Patient ID"].nunique()
```

```
average_age = data["Age"].mean()
average_stay = data["Length of Stay (Days)"].mean()
total_bill = data["Bill Amount (INR)"].sum()

with PdfPages(output) as pdf:
    # Page 1: Executive overview
    fig = plt.figure(figsize=(11.7, 8.3))
    fig.suptitle(
        "Healthcare Data Storytelling - Summary Report",
        fontsize=17,
        fontweight="bold"
    )

    lines = [
        "Synthetic educational dataset - not real patient records.",
        f"Filtered visits: {total_visits:,}",
        f"Unique sample patients: {unique_patients:,}",
        f"Average age: {average_age:.1f} years",
        f"Average length of stay: {average_stay:.1f} days",
        f"Total recorded bill amount: {inr(total_bill)}",
        (
            "Selected date range: "
            f"{data['Visit Date'].min():%d %b %Y} to "
            f"{data['Visit Date'].max():%d %b %Y}"
        ),
        "",
        "Interpretation: These summaries describe only selected",
        "synthetic records. They do not measure quality of care,",
        "prove treatment effectiveness, or provide medical advice."
    ]

    fig.text(
        0.08, 0.82, "\n".join(lines),
        va="top", fontsize=12, linespacing=1.7
    )
    plt.axis("off")
    pdf.savefig(fig, bbox_inches="tight")
    plt.close(fig)

    # Page 2: Monthly visits
    fig, ax = plt.subplots(figsize=(11, 6))
    if not monthly.empty:
        ax.plot(
            monthly["Visit Month"],
            monthly["Visits"],
            marker="o",
            label="Visits"
        )
        ax.set_title("Monthly Hospital Visits")
        ax.set_xlabel("Month")
        ax.set_ylabel("Visit count")
        ax.tick_params(axis="x", rotation=35)
        ax.legend()
        ax.grid(True, alpha=0.25)
    fig.tight_layout()
    pdf.savefig(fig, bbox_inches="tight")
    plt.close(fig)

    # Page 3: Department comparison
    fig, ax = plt.subplots(figsize=(11, 6))
    if not department_summary.empty:
        ax.bar(
            department_summary["Department"],
            department_summary["Visits"]
        )
        ax.set_title("Visits by Department")
        ax.set_ylabel("Visit count")
        ax.tick_params(axis="x", rotation=25)
    fig.tight_layout()
    pdf.savefig(fig, bbox_inches="tight")
    plt.close(fig)
```

```
        # Page 4: Treatment comparison
        fig, ax = plt.subplots(figsize=(11, 6))
        if not treatment_summary.empty:
            ordered = treatment_summary.sort_values("Visits")
            ax.barh(ordered["Treatment"], ordered["Visits"])
            ax.set_title("Visits by Recorded Treatment")
            ax.set_xlabel("Visit count")
        fig.tight_layout()
        pdf.savefig(fig, bbox_inches="tight")
        plt.close(fig)

    output.seek(0)
    return output.getvalue()

# ============================================================
# 5. DASHBOARD HEADER
# ============================================================
st.title("🏥 Healthcare Data Storytelling Dashboard")
st.caption(
    "B.Tech Mini Project | Python • Streamlit • Pandas • Plotly"
)

st.warning(
    "Privacy and safety: The built-in dataset contains fictional sample "
    "records and IDs. Do not upload identifiable patient records. "
    "This academic dashboard is not a clinical decision tool and does "
    "not provide medical advice."
)

# ============================================================
# 6. SIDEBAR: DATA UPLOAD AND FILTERS
# ============================================================
with st.sidebar:
    st.header("Data & Filters")

    uploaded_file = st.file_uploader(
        "Upload healthcare CSV",
        type=["csv"]
    )

    try:
        if uploaded_file is not None:
            raw_data = pd.read_csv(uploaded_file)
            data_source = "Uploaded CSV"
        else:
            raw_data = load_sample_data()
            data_source = "Generated synthetic sample"

        all_data = prepare_data(raw_data)

    except Exception as error:
        st.error(f"Could not load data: {error}")
        st.markdown(
            "**Required columns:** " + ", ".join(REQUIRED_COLUMNS)
        )
        st.stop()

    st.caption(
        f"Source: {data_source} | Valid visits: {len(all_data):,}"
    )

    minimum_date = all_data["Visit Date"].min().date()
    maximum_date = all_data["Visit Date"].max().date()

    selected_dates = st.date_input(
        "Visit date range",
        value=(minimum_date, maximum_date),
        min_value=minimum_date,
        max_value=maximum_date
    )
```

```
    selected_departments = st.multiselect(
        "Department",
        options=sorted(all_data["Department"].unique()),
        default=sorted(all_data["Department"].unique())
    )

    selected_visit_types = st.multiselect(
        "Visit type",
        options=sorted(all_data["Visit Type"].unique()),
        default=sorted(all_data["Visit Type"].unique())
    )

    selected_outcomes = st.multiselect(
        "Outcome",
        options=sorted(all_data["Outcome"].unique()),
        default=sorted(all_data["Outcome"].unique())
    )

# ============================================================
# 7. APPLY FILTERS
# ============================================================
if isinstance(selected_dates, (tuple, list)) and len(selected_dates) == 2:
    start_date, end_date = selected_dates
elif isinstance(selected_dates, (tuple, list)) and len(selected_dates) == 1:
    start_date = end_date = selected_dates[0]
else:
    start_date = end_date = selected_dates

if start_date > end_date:
    start_date, end_date = end_date, start_date

df = all_data[
    (all_data["Visit Date"].dt.date >= start_date)
    & (all_data["Visit Date"].dt.date <= end_date)
    & (all_data["Department"].isin(selected_departments))
    & (all_data["Visit Type"].isin(selected_visit_types))
    & (all_data["Outcome"].isin(selected_outcomes))
].copy()

if df.empty:
    st.warning(
        "No records match the selected filters. "
        "Change the filters in the sidebar."
    )
    st.stop()

# ============================================================
# 8. KEY PERFORMANCE INDICATORS
# ============================================================
total_visits = len(df)
unique_patients = df["Patient ID"].nunique()
average_age = df["Age"].mean()
average_stay = df["Length of Stay (Days)"].mean()
total_bill = df["Bill Amount (INR)"].sum()
average_bill = df["Bill Amount (INR)"].mean()

st.subheader("The healthcare story")
st.markdown(
    "**How many visits are recorded, how are hospital services used, "
    "and what patterns appear across departments and treatments?**"
)

k1, k2, k3, k4, k5 = st.columns(5)
k1.metric("Total Visits", f"{total_visits:,}")
k2.metric("Unique Sample Patients", f"{unique_patients:,}")
k3.metric("Average Age", f"{average_age:.1f} years")
k4.metric("Average Stay", f"{average_stay:.1f} days")
k5.metric("Recorded Bill Total", inr(total_bill))

st.caption(
```

```
    f"Selected period: {start_date:%d %b %Y} - {end_date:%d %b %Y} | "
    f"Average recorded bill: {inr(average_bill)}. "
    "Repeated patient IDs represent multiple visits in this sample, "
    "not necessarily readmissions."
)

# ============================================================
# 9. SHARED DATA SUMMARIES
# ============================================================
monthly = (
    df.groupby("Visit Month", as_index=False)
    .agg(
        Visits=("Patient ID", "size"),
        Unique_Patients=("Patient ID", "nunique"),
        Average_Stay=("Length of Stay (Days)", "mean"),
        Bill_Total=("Bill Amount (INR)", "sum")
    )
)

department_summary = (
    df.groupby("Department", as_index=False)
    .agg(
        Visits=("Patient ID", "size"),
        Unique_Patients=("Patient ID", "nunique"),
        Average_Stay=("Length of Stay (Days)", "mean"),
        Bill_Total=("Bill Amount (INR)", "sum")
    )
)

treatment_summary = (
    df.groupby("Treatment", as_index=False)
    .agg(
        Visits=("Patient ID", "size"),
        Average_Stay=("Length of Stay (Days)", "mean"),
        Average_Bill=("Bill Amount (INR)", "mean")
    )
)

diagnosis_summary = (
    df.groupby("Diagnosis Group", as_index=False)
    .agg(
        Visits=("Patient ID", "size"),
        Average_Stay=("Length of Stay (Days)", "mean"),
        Average_Bill=("Bill Amount (INR)", "mean")
    )
)

# ============================================================
# 10. DASHBOARD TABS - STRUCTURE ADAPTED FROM THE PDF
# ============================================================
tabs = st.tabs([
    "1. Executive Overview",
    "2. Healthcare Visit Trends",
    "3. Department Insights",
    "4. Patient & Treatment Analysis",
    "5. Records & Exports"
])

# ============================================================
# TAB 1: EXECUTIVE OVERVIEW
# ============================================================
with tabs[0]:
    st.header("1. Executive Overview")

    left, right = st.columns(2)

    with left:
        visit_type_summary = (
            df["Visit Type"]
            .value_counts()
            .rename_axis("Visit Type")
```

```
            .reset_index(name="Visits")
        )

        st.plotly_chart(
            px.pie(
                visit_type_summary,
                names="Visit Type",
                values="Visits",
                hole=0.45,
                title="Inpatient vs Outpatient Visits"
            ),
            use_container_width=True
        )

    with right:
        outcome_summary = (
            df["Outcome"]
            .value_counts()
            .rename_axis("Outcome")
            .reset_index(name="Visits")
        )

        st.plotly_chart(
            px.bar(
                outcome_summary,
                x="Outcome",
                y="Visits",
                title="Recorded Visit Outcomes"
            ),
            use_container_width=True
        )

    busiest_department = department_summary.loc[
        department_summary["Visits"].idxmax()
    ]

    st.info(
        f"Healthcare story: **{busiest_department['Department']}** "
        f"has the highest visit volume in this selection "
        f"({int(busiest_department['Visits'])} visits). "
        "Visit volume describes service activity, not care quality "
        "or clinical performance."
    )

    st.markdown(
        "**What this means:** Begin with the overall patient-visit picture. "
        "Then investigate monthly trends, department activity and the "
        "recorded treatment patterns."
    )

# ============================================================
# TAB 2: HEALTHCARE VISIT TRENDS
# ============================================================
with tabs[1]:
    st.header("2. Healthcare Visit Trends")

    st.plotly_chart(
        px.line(
            monthly,
            x="Visit Month",
            y="Visits",
            markers=True,
            title="Monthly Hospital Visit Trend"
        ),
        use_container_width=True
    )

    st.plotly_chart(
        px.line(
            monthly,
            x="Visit Month",
```

```
            y="Average_Stay",
            markers=True,
            title="Average Length of Stay by Month"
        ),
        use_container_width=True
    )

    st.plotly_chart(
        px.bar(
            monthly,
            x="Visit Month",
            y="Unique_Patients",
            title="Unique Sample Patients by Month"
        ),
        use_container_width=True
    )

    peak_month = monthly.loc[monthly["Visits"].idxmax()]
    lowest_month = monthly.loc[monthly["Visits"].idxmin()]

    st.info(
        f"Trend insight: Visit volume is highest in "
        f"**{peak_month['Visit Month']:%B %Y}** "
        f"({int(peak_month['Visits'])} visits) and lowest in "
        f"**{lowest_month['Visit Month']:%B %Y}** "
        f"({int(lowest_month['Visits'])} visits). These patterns can "
        "prompt questions about appointment timing, seasonal demand "
        "and resource planning; this dataset alone cannot establish causes."
    )

    st.dataframe(
        monthly,
        use_container_width=True,
        hide_index=True
    )

# ============================================================
# TAB 3: DEPARTMENT INSIGHTS
# ============================================================
with tabs[2]:
    st.header("3. Department Insights")

    left, right = st.columns(2)

    with left:
        st.plotly_chart(
            px.bar(
                department_summary.sort_values("Visits"),
                x="Visits",
                y="Department",
                orientation="h",
                title="Visit Volume by Department"
            ),
            use_container_width=True
        )

    with right:
        st.plotly_chart(
            px.bar(
                department_summary.sort_values("Average_Stay"),
                x="Average_Stay",
                y="Department",
                orientation="h",
                title="Average Length of Stay by Department"
            ),
            use_container_width=True
        )

    st.plotly_chart(
        px.bar(
            department_summary,
```

```
            x="Department",
            y="Bill_Total",
            title="Recorded Bill Amount by Department"
        ),
        use_container_width=True
    )

    highest_volume_department = department_summary.loc[
        department_summary["Visits"].idxmax()
    ]

    highest_stay_department = department_summary.loc[
        department_summary["Average_Stay"].idxmax()
    ]

    st.info(
        f"Department insight: **{highest_volume_department['Department']}** "
        f"has the highest visit volume "
        f"({int(highest_volume_department['Visits'])} visits). "
        f"**{highest_stay_department['Department']}** has the highest "
        f"average stay ({highest_stay_department['Average_Stay']:.1f} days). "
        "These are operational observations, not measures of department quality."
    )

    st.dataframe(
        department_summary.sort_values("Visits", ascending=False),
        use_container_width=True,
        hide_index=True
    )

# ============================================================
# TAB 4: PATIENT AND TREATMENT ANALYSIS
# ============================================================
with tabs[3]:
    st.header("4. Patient & Treatment Analysis")

    left, right = st.columns(2)

    with left:
        age_summary = (
            df["Age Group"]
            .value_counts()
            .sort_index()
            .rename_axis("Age Group")
            .reset_index(name="Visits")
        )

        st.plotly_chart(
            px.bar(
                age_summary,
                x="Age  Group",
                y="Visits",
                title="Visit Distribution by Age Group"
            ),
            use_container_width=True
        )

    with right:
        gender_summary = (
            df["Gender"]
            .value_counts()
            .rename_axis("Gender")
            .reset_index(name="Visits")
        )

        st.plotly_chart(
            px.pie(
                gender_summary,
                names="Gender",
                values="Visits",
                hole=0.4,
```

```
            title="Visits by Recorded Gender"
        ),
        use_container_width=True
    )

    st.plotly_chart(
        px.bar(
            treatment_summary.sort_values("Visits"),
            x="Visits",
            y="Treatment",
            orientation="h",
            title="Most Frequently Recorded Treatments"
        ),
        use_container_width=True
    )

    st.plotly_chart(
        px.treemap(
            df.groupby(
                ["Diagnosis Group", "Treatment"],
                as_index=False
            ).size(),
            path=["Diagnosis Group", "Treatment"],
            values="size",
            title="Visit Counts by Diagnosis Group and Treatment"
        ),
        use_container_width=True
    )

    most_common_treatment = treatment_summary.loc[
        treatment_summary["Visits"].idxmax()
    ]

    most_common_age = df["Age Group"].mode().iloc[0]

    st.info(
        f"Patient and treatment insight: The most common age band by "
        f"visit count is **{most_common_age}**. "
        f"**{most_common_treatment['Treatment']}** is the most frequently "
        "recorded treatment. Frequency does not indicate treatment "
        "effectiveness or clinical appropriateness."
    )

    st.subheader("Diagnosis group summary")
    st.dataframe(
        diagnosis_summary.sort_values("Visits", ascending=False),
        use_container_width=True,
        hide_index=True
    )

    st.subheader("Treatment summary")
    st.dataframe(
        treatment_summary.sort_values("Visits", ascending=False),
        use_container_width=True,
        hide_index=True
    )

# ============================================================
# TAB 5: RECORDS AND EXPORTS
# ============================================================
with tabs[4]:
    st.header("5. Records & Exports")

    st.dataframe(
        df[REQUIRED_COLUMNS + ["Age Group"]].sort_values(
            "Visit Date",
            ascending=False
        ),
        use_container_width=True,
        hide_index=True
    )
```

```
summary = pd.DataFrame({
    "Metric": [
        "Total visits",
        "Unique sample patients",
        "Average age (years)",
        "Average length of stay (days)",
        "Total recorded bill amount (INR)",
        "Average recorded bill amount (INR)"
    ],
    "Value": [
        total_visits,
        unique_patients,
        round(average_age, 2),
        round(average_stay, 2),
        round(total_bill, 2),
        round(average_bill, 2)
    ]
})

st.subheader("Download reports")

st.download_button(
    "⬇ Download filtered records (CSV)",
    data=df.to_csv(index=False).encode("utf-8"),
    file_name="filtered_healthcare_visits.csv",
    mime="text/csv"
)

st.download_button(
    "⬇ Download KPI summary (CSV)",
    data=summary.to_csv(index=False).encode("utf-8"),
    file_name="healthcare_kpi_summary.csv",
    mime="text/csv"
)

st.download_button(
    "⬇ Download Excel report",
    data=make_excel_report(
        df,
        summary,
        monthly,
        department_summary,
        treatment_summary
    ),
    file_name="healthcare_storytelling_report.xlsx",
    mime=(
        "application/vnd.openxmlformats-officedocument."
        "spreadsheetml.sheet"
    )
)

st.download_button(
    "⬇ Download PDF summary",
    data=make_pdf_report(
        df,
        monthly,
        department_summary,
        treatment_summary
    ),
    file_name="healthcare_storytelling_summary.pdf",
    mime="application/pdf"
)

# PNG export: visits by department.
chart_data = department_summary.sort_values(
    "Visits",
    ascending=False
)
```

```
    fig, ax = plt.subplots(figsize=(8, 4))
    ax.bar(chart_data["Department"], chart_data["Visits"])
    ax.set_title("Hospital Visits by Department")
    ax.set_ylabel("Visit count")
    ax.tick_params(axis="x", rotation=25)
    fig.tight_layout()

    png_output = io.BytesIO()
    fig.savefig(
        png_output,
        format="png",
        dpi=180,
        bbox_inches="tight"
    )
    plt.close(fig)

    st.download_button(
        "⇩ Download department chart (PNG)",
        data=png_output.getvalue(),
        file_name="healthcare_department_visits.png",
        mime="image/png"
    )

# ============================================================
# 11. HEALTHCARE DATA STORYTELLING GUIDE
# ============================================================
st.divider()
st.subheader("How to present the healthcare data story")

st.markdown("""
1. **Question:** How many visits are recorded and which hospital services are used most?
2. **Observation:** Describe the visit, patient, department and treatment patterns shown in the charts.
3. **Meaning:** Explain what the counts and averages show about activity in this selected sample.
4. **Action to investigate:** Authorized hospital teams may review appointment scheduling, staffing and resource planning using validated operational information.
5. **Limit:** Synthetic records cannot establish causes, treatment effectiveness, quality of care or medical recommendations.
""")

st.caption(
    "Academic demonstration only. All built-in records are fictional. "
    "Never use this dashboard to make clinical decisions."
)
```